MEDILINK · PROJECT REPORT

Design and Model

System architecture, data model, algorithms, technology stack, UI design, database design, and system flows of a consent-gated health record.

Source of truth: the implementation in `app/` (Next.js + PostgreSQL + Drizzle)

Verified against code, 92 passing tests, and live end-to-end runs

9 September 2026 · Hosting cost for the pilot: ₹0

1 / 14

1. System Architecture

1.1 Overview

MediLink is a **consent-gated longitudinal health record**. Patients own one chart; verified doctors and labs contribute clinical data; **no professional read is possible without an active, unexpired, scoped consent grant** — except a time-boxed, audited emergency window. Every sensitive action is appended to a **hash-chained audit log**. It is a 3-tier web system, deployed serverless:

```
CLIENT — React Server Components (default) + small client islands
   (forms, drawers, search, theme toggle, notifications bell)
            | HTTPS
APP SERVER — Next.js 16 App Router (Vercel, Node runtime)
   Routes (/portal/*, /api/*, /e/*) · Server Actions (mutations)
   Guards: requireUser → requireRole → requireVerifiedPractitioner
          → access.assert
   Services: consent · records · patients · documents · emergency · audit
   Worker (/api/worker + outbox jobs): extraction, notifications, re-scans
            | SQL (Drizzle ORM)            | signed URLs / bytes
DATA — PostgreSQL                    OBJECT STORAGE (Supabase)
   Supabase (prod) / PGlite             original PDFs/images/text,
   embedded (dev), same schema          immutable, sha-256 hashed
```

1.2 Layers and responsibilities

Layer	Technology	Does	Does NOT do
Presentation	React Server Components; client islands only for interactivity	Renders scoped data; forms via server actions; loading / empty / error / denied states per page	Never enforces security (route guards are UX only)
API boundary	Server Actions + app/api routes	All mutations; every one re-authenticates and re-authorizes	Never trusts client-supplied IDs without ownership checks
Policy engine	access/policy.ts + guard.ts	Fail-closed scope evaluation; scopeForResource() throws on unknown types; every decision logged	Never defaults to allow
Domain services	modules/{consent, records, patients, documents, emergency, audit}	Business rules, transactions, idempotency	Never bypasses the guard
Data	Drizzle ORM → Postgres	Constraints, FKs, unique indexes, hash chain	Never stores originals (metadata + hashes only)

2 / 14

Files	Supabase Storage, per-segment key encoding	Immutable originals, signed-URL reads	Never public buckets
Async	/api/worker + outbox table	Extraction, notifications, re-scans; retries + dead-letter	Never loses a job silently

1.3 Key architectural decisions

1. **Server-first security.** Authorization lives in access.assert() on the server. Hidden buttons and client route guards are convenience only — verified by exercising doctor URLs as a patient (bounced server-side).
2. **Consent is the addressing scheme.** Reads are keyed (practitioner, patient, scopes, time-window), not just (user, row).
3. **Immutable clinical history.** Corrections supersede (new version), never overwrite; disputes flag, never edit.
4. **Deterministic before probabilistic.** Conflict detection and the default summary are pure functions; the LLM is an optional renderer that must cite or drop claims.
5. **Cheap by construction.** One Next.js app + managed Postgres + object storage, all on free tiers. No always-on servers, no per-seat services.
6. **Dev/prod parity.** PGlite runs the same Drizzle schema and migrations locally, so data-layer behavior (including the audit chain) is identical in both environments.

3 / 14

2. Data Model

2.1 Entity map

```
user ──1:1── patients ──1:N── consents ──N:1── practitioners ──N:1── user
  │              │                │                │
  │              │                │                └── organization membership
  │              ├── conditions / allergies / medication_statements
  │              ├── observations / procedures / immunizations
  │              ├── encounters / diagnostic_reports / documents
  │              ├── clinical_resource_versions (immutable seals)
  │              ├── data_conflicts (system-detected)
  │              └── data_disputes (patient-raised)
  │
  ├── access_requests (doctor asks → patient narrows → grant/decline)
  ├── emergency_windows (break-glass, 60 min, step-up authed)
  ├── notifications (inbox per user)
  └── audit_events (hash-chained, append-only)
```

2.2 Core entities

Entity	Stores	Why it matters
user	Better-Auth identity + role (patient / doctor / org_staff / org_admin / platform_admin)	Role changes are privileged and audited
patients	Demographics, unique patientCode (ML-XXXXX-XXXXX-C), optional 14-digit ABHA, blood-group self-report; userId when claimed, createdById before that	The chart root; code possession alone grants nothing
practitioners	One row per clinician (UNIQUE userId): registration no., council, specialty, verificationStatus, verifiedAt/By	Resubmission updates in place so grants (keyed by practitioner id) survive
organizations	Facilities (e.g. Sunrise Diagnostics) with verification state	Staff act under the facility's verified status
consents	patient, grantee, purpose, scopes[], expiresAt, revocation flag, content hash + receipt code	Revocation is immediate — every read re-evaluates
access_requests	Proposed scopes → patient-narrowed grant, or decline	No pre-selected scopes; patient can only narrow, never widen
emergency_windows	One live window per practitioner+patient, 60 min, step-up authed	Critical scopes only; notified; admin-reviewed

4 / 14

documents	Metadata of originals: title, type, sha-256, storage key, attestation	Bytes live in object storage, never in the DB
Clinical tables ×9	patientId, clinical fields, trust (verified / extracted / self_reported), source/author refs, supersession pointers	Never deleted on correction — only superseded
clinical_resource_versions	Immutable seal per write: canonical payload + sha-256, lifecycle (active / corrected / superseded / disputed / entered_in_error)	Append-only history with correction links
data_conflicts	Engine findings with content _key, open/resolved, resolution notes	Evidence preserved on resolution
data_disputes	Patient flags with reason; clinician confirm/dismiss + notes	Version flag lifts when nothing open references the resource
search_chunks	Per-row display label + full-text content (GIN index)	Powers search and dispute labels
notifications	Per-user inbox (grants, requests, emergency alerts)	Durable via outbox
audit_events	actor, action, resource, prevHash → hash chained (GENESIS first)	Periodic checkpoints let anyone re-verify the chain

2.3 Relationship rules that matter

- Grants reference `practitioner.id` (stable across resubmission), never the user id.
- Dispute raising verifies the flagged row belongs to the raiser's chart (single UNION ALL over the 9 clinical tables) — cross-patient flagging is rejected.
- `conflicts.resolved_by` has a user FK: human resolutions reference a user; system auto-resolutions use NULL (no fake "system" user).
- Deletions cascade only downward (search chunks die with their row). Clinical facts are never hard-deleted.

5 / 14

3. Algorithm Selection

Every algorithm was chosen for **auditability over cleverness** — a health record must be explainable in a dispute.

3.1 Contradiction detection (deterministic rules)

Detector	Rule	Severity
Blood group	Verified lab value ≠ patient self-report; verified value displayed	critical
Allergy	Same substance, distinct criticality‖reaction across live rows	warning
Duplicate document	Identical contentHash on 2+ documents	info
Lab value	Same analyte + same date, distinct numeric values	warning

Each row carries a content `_key` (sha-256 of kind + details) so distinct analytes/allergies coexist and re-scans dedupe. **Reconciliation:** after every scan, open conflicts whose key was not reproduced are auto-closed with a preserved note — corrections and supersessions clear stale banners automatically. **Why rules, not ML:** reproducible, unit-testable, zero hallucinations, millisecond runtime.

3.2 Summarization (deterministic SBAR + cited LLM option)

Default mode is a **pure function**: scoped facts → SBAR structure (key events, history, recent changes, open data issues), every sentence carrying source citations. LLM mode is a renderer under the same contract: **a claim without a source is dropped**, recorded-fact language never implies AI judgment, and the mode badge (deterministic vs LLM · model) is always shown. A stored summary from broader consent is never shown to a narrower grantee.

3.3 Identity and access algorithms

- **Patient code** `ML-XXXXX-XXXXX-C` : random segments plus a check character that catches typos; case/dash-insensitive normalization. Possession grants nothing.
- **Policy evaluation:** `scopeForResource()` maps every record kind to its consent scope and **throws on unknown kinds** (fail-closed); emergency basis exposes a fixed critical set; revocation re-evaluated per read; every decision logged.
- **Rate limiting** on discovery/lookup endpoints against enumeration.
- **Verification:** mock provider auto-approves plausible registrations (demo); implausible ones queue for human review; every submission/decision audit-logged.

6 / 14

3.4 Retrieval

Record search runs over `search_chunks` (label + full text, GIN-indexed), always filtered by patient and by the reader's granted scopes; the same index supplies human labels for the concerns queue.

7 / 14

4. Technology Stack

Layer	Choice	Why (and cost)
Framework	Next.js 16 (App Router, RSC, Server Actions)	One deployable for SSR, mutations, APIs; server components keep PHI out of client bundles. Free.
Language	TypeScript (strict, 0 errors)	Catches scope/data mismatches at compile time, under 92 tests.
Styling	Tailwind CSS v4 (@theme inline tokens)	Themeable design tokens; no runtime CSS-in-JS cost.
ORM	Drizzle ORM	Readable SQL, file migrations, identical schema dev↔prod.
Database	PostgreSQL — Supabase (prod) / PGlite embedded (dev)	Managed Postgres free tier; zero-setup identical engine locally.
Object storage	Supabase Storage (signed URLs)	Private buckets; per-segment key encoding. Free tier covers a pilot.
Auth	Better-Auth (email/password, sessions, reset, verification)	Self-hosted sessions in our own DB; OAuth boundary ready. ₹0.
Hosting	Vercel Hobby	Serverless Next.js + worker route; 60s maxDuration for uploads. Free.
Tests	Vitest + Playwright + axe-core	92 unit + ~60 browser checks + contrast/dark audits. Free.
Icons	Lucide	One consistent icon language.
Removed	three.js / react-three-fiber	Rendered poorly; cost bundle + warnings; replaced with CSS mocks.

> **Total hosting cost for the pilot: ₹0** — Vercel Hobby + Supabase Free, with no per-seat or per-API-key service in the critical path.

8 / 14

5. User Interface (UI) Design

5.1 Design language

- **Typography:** Fraunces (display serif — editorial trust), Instrument Sans (UI), Plex Mono (codes, hashes, values).
- **Color:** pine (primary trust signal), alert/amber/info statuses — every pair contrast-tested per mode (WCAG AA, axe-enforced). Glass effects only where they aid hierarchy; body-text cards are solid.
- **Shape & motion:** 12–24px radii, 1px line borders, soft shadows; one Reveal scroll system (opacity/translate only) with full prefers-reduced-motion support and a view-transition theme wipe.

5.2 Themes

Daylight (default), Obsidian (dark), system-follow, plus an independent Reduce Transparency toggle — persisted, with a pre-hydration script so there is no flash. Dark mode is a separately designed palette, not an inversion (verified across 12 pages).

5.3 Information architecture per role

Surface	Contents
Marketing (/)	Thesis hero with live-pattern consent card, trust strip, evidence pillars, chain section (seal → gate → citation), 4-step journey, role cards, "what we do NOT do" honesty panel, FAQ, CTA. Zero canvas, mobile-first.
Patient portal	Dashboard glance band (allergies / problems / meds / activity), health record with provenance drawers ("flag a concern"), documents, timeline, access centre (grant / narrow / decline / revoke), emergency QR card + print, access history, profile + ABHA + themes.
Doctor workspace	Identifier lookup (never a name/phone directory); AccessBanner (purpose, scopes, exact expiry, renewal); patient context bar (wrong-patient escape); 10-second glance band; cited evidence summary; concerns queue (confirm/dismiss); labs & trends; documents; contribute; inbox; emergency access.
Lab console	Consented-patient upload (original preserved byte-for-byte, values extracted to trends), attestation review queue.
Admin	Verification queues, emergency oversight (reviewed/unreviewed + reviewer notes), audit stream, roles.

5.4 AI UX

Summaries render as an **evidence rail**: recorded-fact groups as stations on a line, each claim with inline citation chips resolving to sealed rows — visually distinct from authoritative records, with mode badge, generation timestamp, and a "verify against sources" affordance. Long responses paginate by group.

9 / 14

Errors, empty states (honest "no record found as of this date"), and permission-denied states are all designed, not defaults.

5.5 Every-state discipline

Each workflow ships normal / loading / empty / error / partial-data / denied / expired-session / malformed-input / slow-network / mobile / both-themes — enforced by the 34-check Playwright suite, not by convention.

10 / 14

6. Database Design

6.1 Schema surfaces (Drizzle, src/server/db/schema.ts)

Table	Key columns / constraints	Indexes
user	Better-Auth fields + role	id, email unique
patients	patientCode unique, validated abha, userId?, createdById	patientCode unique, userId
practitioners	userId UNIQUE, registration no., council, verificationStatus, verifiedAt/By	practitioner_user_unique, status
organizations + staff	Verification state	status
consents	patient, grantee, purpose, scopes[], expiresAt, revoked, hash, receipt	patient+status, grantee, expiry
access_requests	Suggested vs granted scopes, state, decline reason	patient+state
emergency_windows	practitioner+patient, 60-min liveness, reason, step-up proof, review	live-window lookup
documents	patient, hash, storage key, attestation	patient, hash
clinical tables ×9	patientId FK, trust, source/author refs, supersession pointers	patient per table
clinical_resource_versions	resourceId, canonical payload, sha-256, lifecycle	resource lookup
data_conflicts	patient, kind, _key in details, status	(patient, status)
data_disputes	patient, resource, reason, status, resolution + notes	patient+status
search_chunks	patient, recordId, label, content	GIN full-text, patient
notifications	user, kind, payload, read flag	user+read
audit_events	actor, action, resource, prevHash → hash	actor, patient, action
practitioner_verifications	Per-submission history (provider, outcome)	practitioner

6.2 Integrity mechanisms

- **Fail-closed reads:** every clinical query is built with the evaluated scope set — narrower consent means fewer tables touched, never client-side row filtering.
- **Idempotency:** extraction retry deletes by sourceDocumentId before re-insert; break-glass arming is idempotent per live window; outbox dedupes by event key.

11 / 14

- **Concurrency:** audit hash computed in-transaction (single pgcrypto statement in prod; serialized fallback in tests); corrections link replacesVersionId, so concurrent edits fork visibly instead of clobbering.
- **Migrations:** numbered Drizzle SQL + journal, applied identically to Supabase and PGlite. Seed produces the golden chart, conflicts, SBAR summary, and demo accounts ending with chain intact ✓.

12 / 14

7. System Flow

7.1 Consent lifecycle (the core loop)

```
Doctor (verified, no access)
  │ identifier lookup (ID only — nothing leaks pre-consent)
  ▼
Access request (proposes scopes + purpose + duration)
  │ patient inbox card: who / why / which categories / how long
  ├─ Decline ──▶ closed + notified
  └─ Grant (patient NARROWS scopes) ──▶ consent {hash, receipt, expiry}
        │ doctor workspace unlocks ONLY granted scopes
        │ AccessBanner shows purpose · scopes · exact expiry
        ├─ Renew ──▶ fresh request before expiry
        └─ Revoke ──▶ immediate; next read denied + audited
```

7.2 Lab contribution → citation

```
Lab uploads original (consented patient only)
  → magic-byte validation, ≤ 4 MB → bytes stored immutable + sha-256
  → worker extracts observations → structured rows (trust = extracted)
  → attestation (doctor/lab reviews, seals a version)
  → contradiction re-scan → indexed into search_chunks
  → summary cites it (claim ↔ sealed row, page-level)
```

7.3 Emergency (break-glass)

```
Unresponsive patient → verified doctor opens Emergency access
  → enters Patient ID + clinical reason + own password (step-up)
  → ONE 60-minute window (idempotent) → critical scopes only
  → every read audited → patient notified → admin reviews afterwards
```

7.4 Dispute loop

```
Patient flags a row in a provenance drawer (chart-ownership checked)
  → version marked disputed → doctor "concerns" card appears
  → Confirm (issue recorded; correction follows separately)
     or Dismiss (note required for accountability)
  → last-open resolution lifts the version flag;
     full trace kept in disputes table + audit
```


13 / 14

7.5 Discovery & scan paths

```
Handoff token (single-use, expiring) → redeem → request → grant
QR scan (/e/<token>) → login (context preserved via ?next=) → scoped view
Unknown token → invalid state, zero patient data
```

Where this document and the code disagree, the code wins.

14 / 14